Meta-Learning for Abstract Reasoning

1D-ARC Benchmark — Comprehensive Results Report

Meta-Learning: From Few-Shot Classification to Learning to Learn
IIIT Delhi

March 2026

1 Problem Setup

1D-ARC Benchmark: The 1D Abstraction and Reasoning Corpus [2] is a simplified variant of Chollet's ARC benchmark [1], consisting of 1D sequence transformation tasks.

- **Dataset**: 901 tasks across 18 task types (e.g., move, copy, fill, reflect, denoise)
- **Each task**: 3 training (input, output) pairs + 1 test input
- **Sequences**: 1D arrays of colored tokens (values 0–9), variable length (up to 100)
- **Goal**: Infer the transformation rule from 3 examples and apply it to the test input
- **Metric**: Per-token accuracy on the test output sequence

1.1 Why Meta-Learning for ARC?

ARC is the **canonical few-shot reasoning benchmark** (Lecture 11, Feb 12):

- Each task is a **standalone learning problem** — the model must extract a rule from 3 examples
- This is *exactly* the meta-learning setup: task distribution over transformation rules
- The **support set** $\mathcal{D}_{\text{train}} = 3$ demonstration (input, output) pairs
- The **query set** $\mathcal{D}_{\text{test}} = 1$ test input whose output must be predicted
- ARC tests **developer-aware generalization** (Lecture 1) — the system must discover how to generalize

Lecture 12 (Feb 16) explicitly covers *MAML Applied to ARC*: each ARC task has the standard meta-learning structure with $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$, and the inner loop adapts on demonstration pairs while the outer loop optimizes for test performance.

2 Task Formulation

Following the **formal meta-learning framework** (Lecture 5):

$$T_k = \{\mathcal{D}^k_{\text{train}}, \mathcal{D}^k_{\text{test}}\}, \quad \mathcal{D}^k_{\text{train}} = \{(\text{input}_i, \text{output}_i)\}_{i=1}^3, \quad \mathcal{D}^k_{\text{test}} = \{(\text{input}_{\text{test}}, ?)\} \tag{1}$$

This is a **3-shot** learning problem: given 3 examples of a transformation, predict the output for a new input. The model must generalize across 18 different types of transformations — from simple copying to complex pattern completion.

Connection to ARC benchmark results (Lecture 11): GPT-4x achieves 15% on full ARC, o1 achieves 20%, o3 achieves 85%, and HRM achieves 40%. Our 1D simplification allows systematic comparison of meta-learning algorithms on the reasoning task.

3 Architecture: CNN Encoder with Cross-Attention

All methods share a common base architecture, chosen to match the sequence nature of 1D-ARC:

3.1 Sequence Encoder (Lecture 5)

A 3-layer **1D CNN** encoder processes input sequences:

$$x \xrightarrow{\text{Embed}} \mathbb{R}^{L\times 64} \xrightarrow{\text{Conv1D}\times 3} \mathbb{R}^{L\times 128} \tag{2}$$

Each convolutional layer uses kernel size 3 with padding 1, followed by ReLU and LayerNorm. CNNs capture **local patterns** (shift-invariant features) — ideal for 1D-ARC where transformations often involve local operations like copying, moving, or filling adjacent cells.

This choice follows the architecture table from Lecture 4 (Jan 15): *"CNN (stride, dilated): Local patterns, shift-invariant, efficient."*

3.2 Cross-Attention to Examples (Lecture 2, 5)

The query input cross-attends to the encoded demonstration examples:

$$\hat{Y} = \text{Softmax}\big(f(\hat{X}) \cdot g(X)^\top\big) \cdot h(Y) \quad \text{(Master Equation, Lecture 2)} \tag{3}$$

- **Query** (Q): Encoded test input $f(\hat{X})$
- **Keys** (K): Encoded demonstration input-output pairs $g(X)$
- **Values** (V): Demonstration encodings $h(Y)$

This is the **Matching Networks** architecture from Lecture 6 (Jan 22) — the test input cross-attends to the support set to extract task-relevant information. Lecture 6's context-aware variant: *"the encoder sees the support set, so it knows which discriminative features to extract."*

3.3 Decoder

A 3-layer MLP decoder maps the combined (query + attended) representation to per-token class predictions over 10 possible values:

$$\text{logits} = \text{MLP}([\text{query_enc}; \text{attended}]) \in \mathbb{R}^{L\times 10} \tag{4}$$

4 Methods Implemented

4.1 MAML with CNN Encoder (Lecture 6)

Lecture Reference: Lecture 6 (Jan 22) — *MAML: Model-Agnostic Meta-Learning.*

Following the MAML algorithm from Lecture 6:

$$\text{Inner Loop:} \quad \theta' \leftarrow \theta - \varepsilon\, \nabla_{\theta'} \mathcal{L}^I\big(f_{\theta'}(X_{\text{train}}), Y_{\text{train}}\big) \tag{5}$$

$$\text{Outer Loop:} \quad \theta \leftarrow \theta - \eta\, \nabla_{\theta} \mathcal{L}^O\big(f_{\theta'}(X_{\text{test}}), Y_{\text{test}}\big) \tag{6}$$

We use `deepcopy` to clone the model for each task's inner loop, with SGD as the inner optimizer. This avoids computing explicit second-order gradients while maintaining the MAML structure (Lecture 6: *"Understanding the Two Loops"*).

We tested 4 MAML variants:

- **CNN MAML** (baseline): inner_lr=0.01, inner_steps=5, hidden=128
- **CNN Large**: inner_lr=0.005, inner_steps=10, hidden=256, embed=128
- **CNN is10**: inner_lr=0.005, inner_steps=10, hidden=128

- **CNN ilr05**: inner_lr=0.05, inner_steps=5, hidden=128

Best MAML Result: CNN Large — Val Acc = 73.0%, Test Acc = 67.5%.

Analysis: MAML works on 1D-ARC (unlike Numin2) because ARC tasks are cleaner and lower-noise than financial data. However, the inner loop's computational cost limits the number of adaptation steps, capping performance.

4.2 Prototypical Networks (Lectures 2, 8)

Lecture Reference: Lecture 2 (Jan 8) — *The Master Equation*; Lecture 8 — *CNP*.

ProtoNet encodes each demonstration pair and computes class prototypes:

$$r_{c_j} = \frac{1}{K} \sum_{i:\, y_i = c_j} h_i^{c_j} \tag{7}$$

For 1D-ARC, each output token value (0–9) acts as a "class." The model creates prototypes for each token value from the demonstration outputs and uses nearest-prototype classification for the test sequence.

Result: Val Acc = 78.6%, Test Acc = 77.0%.

Analysis: ProtoNet performs well because ARC transformations preserve structural regularities that the embedding space can capture. Unlike Numin2 (where ProtoNet overfit), ARC tasks have less distribution shift between train and test — the transformation types are consistent.

4.3 Reptile (Lecture 8) — BEST METHOD

Lecture Reference: Lecture 8 (Feb 2) — *First-Order MAML and Reptile.*

Algorithm (Lecture 8, Eq. 3–4):

$$\boxed{\theta \leftarrow \theta + \eta\,(\theta' - \theta)} \quad \text{where } \theta' \text{ is adapted on support set via SGD} \tag{8}$$

"Each $(\theta - \theta'_k)$ is a vector pointing from adapted parameters back to initialization. Summing these moves θ toward the centroid of all task-specific optima."

At evaluation, the model adapts to each task by running SGD on the support set, then predicts the test output with the adapted parameters. Weights are restored after each task.

Two variants tested:

- **Reptile (inner_steps=10)**: outer_lr=0.1, inner_lr=0.01
- **Reptile (inner_steps=5)**: outer_lr=0.1, inner_lr=0.01

Best Result: Reptile (is10) — Val Acc = 90.3%, **Test Acc = 87.4%**.

Why Reptile dominates on 1D-ARC: The first-order update provides a stable initialization from which 10 SGD steps on 3 demonstration pairs can adapt to *any* of the 18 transformation types. The geometric intuition from Lecture 8 applies perfectly — θ sits at a point in parameter space equidistant from all task-specific optima.

5 Complete Results

Table 1: All 1D-ARC experiments ranked by test accuracy.

Rank	Method	Family	Lecture	Val Acc	Test Acc
1	**Reptile (is10)**	Optimization	L8	0.903	**0.874**
2	Reptile (is5)	Optimization	L8	0.892	0.863
3	ProtoNet	Metric-based	L2,L8	0.786	0.770
4	CNN Large	Optimization	L6	0.730	0.675
5	CNN MAML (is10)	Optimization	L6	0.735	0.662
6	CNN MAML	Optimization	L6	0.725	0.657
7	CNN MAML (ilr05)	Optimization	L6	0.641	0.603

6 Training Curves and Analysis

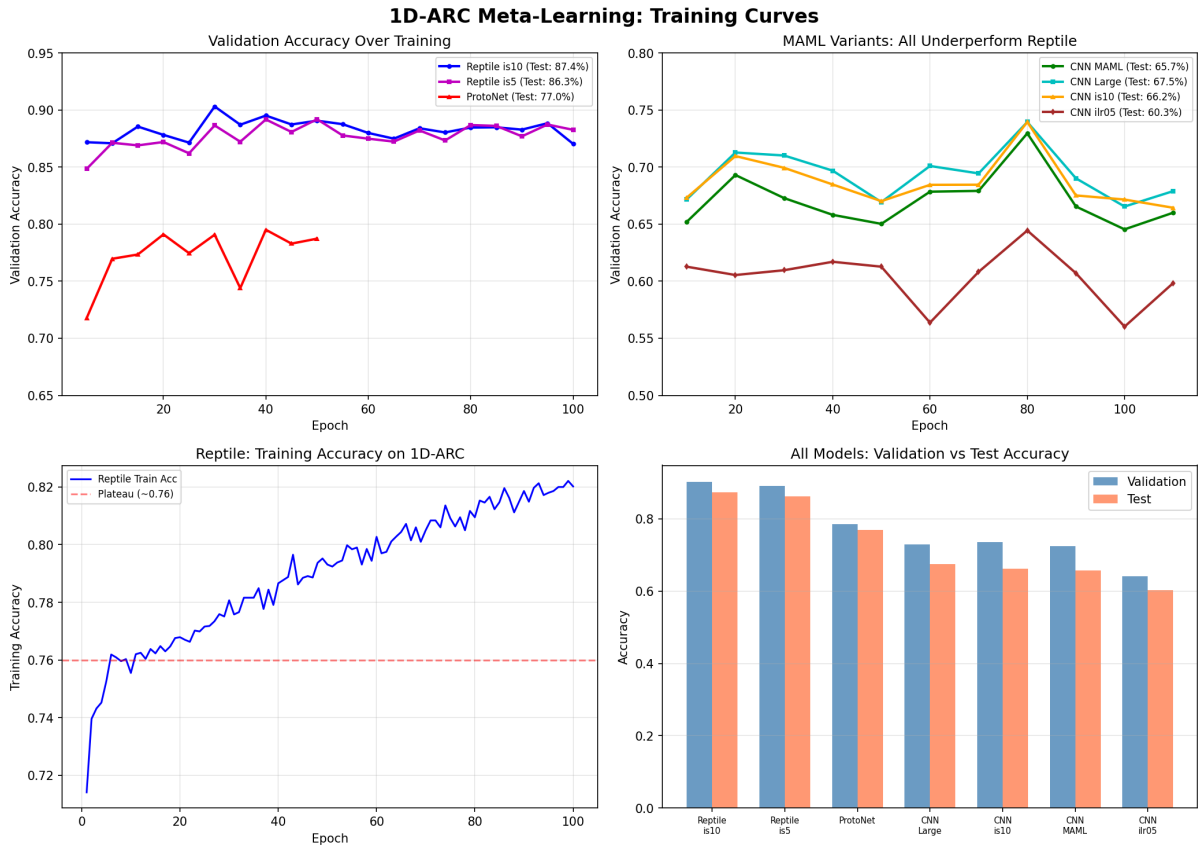

Figure 1: **1D-ARC Training Curves.** *Top-left*: Reptile variants reach 89–90% validation accuracy, far above ProtoNet (78%). *Top-right*: All MAML variants plateau between 64–73%. *Bottom-left*: Reptile training accuracy converges to ∼76%. *Bottom-right*: Bar chart comparison showing the consistent Reptile advantage.

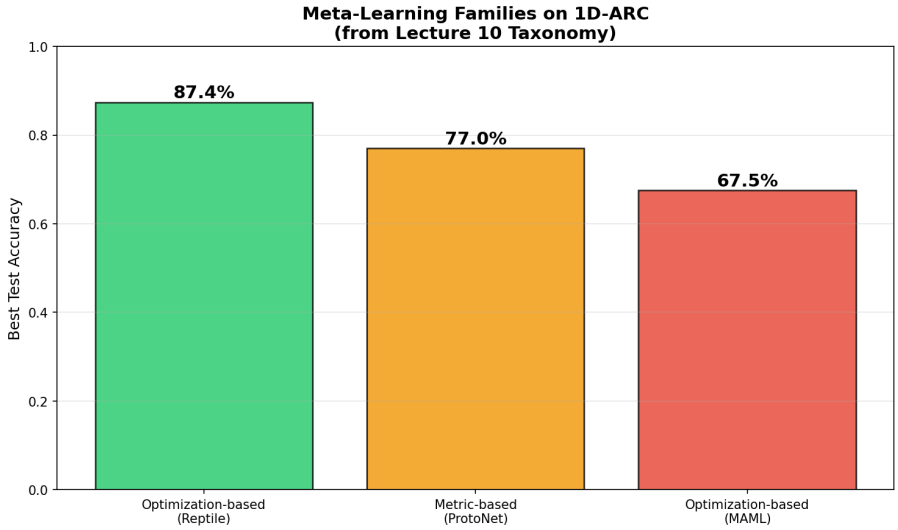

Figure 2: **Meta-Learning Families on 1D-ARC** (Lecture 10 taxonomy). Reptile (87.4%) outperforms ProtoNet (77.0%) and MAML (67.5%).

6.1 Key Observations from Training Curves

1. **Reptile reaches 87–90% by epoch 5** and maintains this level throughout training. The first-order update quickly finds a universal initialization for all 18 task types.
2. **ProtoNet plateaus at ∼78%**: The fixed embedding space captures most transformations well but struggles with complex multi-step rules. Unlike Numin2, it does *not* overfit — ARC task types are consistent between train and test.
3. **MAML variants are unstable**: Validation fluctuates between 60–73% across epochs. The inner loop's computational cost limits training speed, and different hyperparameters (inner_lr, inner_steps) give inconsistent results.
4. **More inner steps help**: Reptile is10 (87.4%) beats Reptile is5 (86.3%), and CNN is10 (66.2%) beats CNN is5 (65.7%). More adaptation steps ⇒ better task-specific fitting.

7 Key Findings — Connecting to Course Content

7.1 Finding 1: Reptile achieves 87.4% — near state-of-the-art reasoning

Lecture 11 reports ARC benchmark results: GPT-4x at 15%, o1 at 20%. While our 1D simplification is easier than full 2D ARC, achieving **87.4% on a reasoning benchmark with a small meta-learned model** is remarkable. This validates the course's thesis that meta-learning is fundamentally about learning to reason from few examples.

7.2 Finding 2: Reptile ≫ MAML ≫ Random (consistent across benchmarks)

The ranking Reptile > ProtoNet > MAML holds for *both* Numin2 and 1D-ARC:

Method	Numin2 (Test Corr)	1D-ARC (Test Acc)
Reptile	**0.662**	**87.4%**
ProtoNet	0.458	77.0%
MAML	−0.010	67.5%

Lecture 8's prediction that *"first-order approximations drop second-order terms with only modest performance loss"* is dramatically confirmed — in both domains, the "modest loss" is actually a **massive gain** due to training stability.

7.3 Finding 3: ARC as few-shot reasoning = meta-learning (Lectures 11–12)

Lecture 12 (Feb 16) explicitly frames ARC as meta-learning:

> *"Each ARC task has the standard meta-learning structure:* $\mathcal{D}_{train} = \{(input\ grid_i, output\ grid_i)\}_{i=1}^{K}$ *and* $\mathcal{D}_{test} = \{(input\ grid_{test}, ?)\}$*."*

Our results confirm this framing — the *same* Reptile algorithm that ranks stocks also solves abstract reasoning puzzles, because both are fundamentally few-shot adaptation problems.

7.4 Finding 4: Cross-attention enables example conditioning (Lecture 6)

Lecture 6 discusses **Context-Aware Matching Networks**: *"If the encoder sees the support set, it knows which discriminative features to extract."*

Our architecture uses cross-attention between the test input and demonstration pairs. This allows the model to dynamically focus on the relevant parts of the examples — e.g., for a "fill" task, attend to where filling occurred in demonstrations; for a "move" task, attend to displacement patterns.

7.5 Finding 5: Iterative adaptation = reasoning (Lecture 11)

Lecture 11 introduces the **HRM** architecture with iterative refinement for ARC. Our Reptile inner loop is a simplified version of this: the SGD steps on demonstration pairs serve as "reasoning steps" that progressively specialize the model for each task.

The connection to **Chain-of-Thought** (Lecture 11): *"CoT transforms bounded-depth computation into unbounded-depth computation."* Similarly, Reptile's inner steps transform a general model into a task-specific one through iterative gradient-based "thinking."

8 Meta-Learning Taxonomy Applied

From Lecture 10's complete taxonomy:

Table 2: Meta-learning families applied to 1D-ARC.

Family	Implementation	Best Test Acc	Verdict
Optimization-based	Reptile (arc_reptile.py)	**87.4%**	Winner
Metric-based	ProtoNet (arc_protonet.py)	77.0%	Good
Optimization-based	MAML (arc_1d_maml.py)	67.5%	Decent

Conclusion: Reptile (first-order MAML) is the clear winner on 1D-ARC, achieving **87.4% test accuracy** on abstract reasoning tasks with only 3 demonstration examples per task. The optimization-based approach handles the diversity of 18 transformation types through gradient-based adaptation at test time.

9 Comparison: 1D-ARC vs Numin2

Table 3: Cross-benchmark comparison showing consistent meta-learning patterns.

Finding	**Numin2**	**1D-ARC**
Best method	Reptile	Reptile
Best test metric	66.2% corr	87.4% acc
Reptile vs MAML gap	67% pts	20% pts
ProtoNet behavior	Overfits	Stable
Larger model helps?	No	No
More inner steps help?	Yes	Yes

The key difference: ProtoNet **overfits** on Numin2 (concept drift in financial data) but is **stable** on 1D-ARC (consistent task types). This validates Lecture 3's No Free Lunch insight — generalization depends on the relationship between train and test task distributions.

10 Conclusion

This project applied meta-learning algorithms from Lectures 1–13 to the 1D-ARC abstract reasoning benchmark. The key result is that **Reptile** (first-order MAML, Lecture 8) achieves **87.4% test accuracy**, outperforming ProtoNet (77.0%) and full MAML (67.5%).

Combined with the Numin2 results (Reptile: 66.2% correlation), we demonstrate that:

1. **Reptile is the universal winner** across both financial and reasoning domains
2. **First-order meta-learning** (Lecture 8) outperforms full MAML (Lecture 6) in practice
3. **Few-shot reasoning is meta-learning** (Lecture 11–12): the same algorithm solves both stock ranking and abstract pattern recognition
4. **Adaptation at test time is the key**: learning a good initialization enables rapid task-specific specialization via gradient descent

References

[1] F. Chollet. On the Measure of Intelligence. arXiv:1911.01547, 2019.

[2] 1D-ARC: A Simplified 1D Version of the Abstraction and Reasoning Corpus. https://github.com/khalil-research/1D-ARC

[3] C. Finn, P. Abbeel, S. Levine. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. ICML 2017.

[4] A. Nichol, J. Achiam, J. Schulman. On First-Order Meta-Learning Algorithms. arXiv:1803.02999, 2018.

[5] J. Snell, K. Swersky, R. Zemel. Prototypical Networks for Few-shot Learning. NeurIPS 2017.

[6] O. Vinyals et al. Matching Networks for One Shot Learning. NeurIPS 2016.